ECS: 面向显存受限设备的专家协调系统

ECS: An Expert Coordination System for Memory-Constrained MoE Inference

摘要

在 4 GB 显存的笔记本 GPU 上运行一个需要 9.65 GiB 存储的 MoE 模型，听起来像是 OOM 的必然结局。
ECS 说明这在四个层面同时重构推理栈的前提下是可行的——代价是 2.64–3.01 tok/s 的解码速度。
本文报告这套系统的设计、实测结果，以及在该约束下**失效**的三项优化。

混合专家（MoE）模型的参数量与其激活量之间存在数量级差距，理论上适合在显存受限的设备上运行；但把这一理论优势落地，需要在权重供给、KV 管理、请求调度和跨设备协同四个层面同时重构推理栈。
本文报告 ECS（Expert Coordination System）的设计、实现与实测结果。ECS 是一个非官方扩展，建立在 DeepInfer 的 model_executor 之上，目标平台是单张 4 GB 笔记本 GPU，以及后续的双卡 RTX 3080 Ti。

在单卡 4 GB 上，ECS 以 CPU/GPU 分级存储和流式专家供给，使 9.65 GiB 的 DeepSeek-V2-Lite Q4_K_M
在 2.23 GiB 峰值显存下完成推理（2.64–3.01 tok/s）；同一套内核在 Qwen2.5-3B 上达到 42.7 tok/s 稳态。
在双卡配置上，ECS 将路由专家下放到第二张卡，远端 rank 不加载 transformer，
峰值占用仅 38 MiB，并在 8 个并发请求下达到 25.90 tok/s 的聚合解码吞吐（单卡的 6.17 倍）。

本文同时报告三类**否定性结果**，我们认为它们对该方向的研究者同样有价值：
（1）paged KV 的收益严格依赖上下文长度，在 17-token prompt 上与整块缓存无法区分；
（2）prefill/decode 重叠在 NCCL 上不可实现，根因是一个 process group 的点对点通信共享单条内部流，
而非调度器设计；（3）以专家为单位的分发在单 worker 下会退化为串行队列，
复制 worker 是提高上限的必要条件，但受硬件限制未能验证。

关键词：混合专家模型；显存受限推理；KV 缓存分页；专家并行；NCCL

> **本稿状态**：工作稿（work in progress）。横向基线已补测且**不利于 ECS**（llama.cpp CPU 为 23.7–25.5 tok/s、386 MiB，ECS 为 2.64–3.01 tok/s、2.23 GiB，§5.6 G1）。
> 本文不主张任何性能优势，只主张可行性、正确性、失效条件与方法论记录。
> 重复测量与双卡显存混淆（G2、G3）仍未补齐。

Abstract

Running a 16B MoE whose checkpoint occupies 9.65 GiB on a 4 GB laptop GPU sounds like a guaranteed OOM.
ECS shows it is feasible once the inference stack is rebuilt at four layers at once -- at 2.64-3.01 tok/s.
We report the design, the measurements, and the three optimisations that **do not** work under this constraint.

Mixture-of-Experts (MoE) models separate total parameters from activated parameters by orders of
magnitude, which in principle suits memory-constrained hardware. Realising that advantage requires
rebuilding the inference stack at four layers simultaneously: weight supply, KV management, request
scheduling, and cross-device coordination. This report describes ECS (Expert Coordination System),
an unofficial extension built on DeepInfer's model_executor, targeting a single 4 GB laptop GPU and,
later, a pair of RTX 3080 Ti cards.

On one 4 GB device, tiered CPU/GPU storage with streamed expert supply runs the 9.65 GiB DeepSeek-V2-Lite Q4_K_M checkpoint at 2.23 GiB peak (2.64-3.01 tok/s), while the same kernels reach
42.7 tok/s steady state on Qwen2.5-3B. On two cards, ECS places routed experts on the second device
without loading the transformer there (38 MiB peak) and reaches 25.90 tok/s aggregate decode at eight
concurrent requests, 6.17x the single-card figure.

We also report three negative results that we believe are useful to others pursuing this direction: paged KV only pays off as context grows, prefill/decode overlap is blocked by NCCL's single internal
communication stream rather than by scheduler design, and per-expert dispatch degenerates into a serial
queue with a single worker.

Keywords: MoE; memory-constrained inference; paged KV cache; expert parallelism; NCCL

---

1. 引言

MoE 的卖点是"参数量大、激活量小"。DeepSeek-V2-Lite 有 15.7B 参数，但每个 token 只激活其中一小部分。
按这个逻辑，它应该能塞进比 15.7B 小得多的显存里。

实际做起来，这个"应该"卡在四个地方：

1. **权重供给**：即使只激活一部分，权重文件本身有 9.65 GiB（Q4_K_M），不可能整体装进 4 GB。
2. **KV 管理**：MoE 只压缩了 FFN，attention 的 KV 仍然随上下文线性增长。
3. **请求调度**：单请求跑通不等于能服务多个请求；批处理会引入新的内存压力。
4. **跨设备协同**：想把专家摊到多张卡上，就要解决激活值交换、负载均衡和故障恢复。

现有工作大多假设显存充足（≥24 GB），在这些假设下 MoE 推理已有成熟方案。
本文的问题是：**当显存预算比模型小一个数量级时，最小可行的推理栈长什么样?**

我们的定位是工程报告而非算法创新。三个贡献：

据我们所知，ECS 是第一个在 ≤4 GB GPU 显存的约束下，同时实现权重流式供给、paged KV 管理、连续批处理与跨设备专家分片的端到端开源推理栈，并系统性地报告了该约束下的**优化收益边界与失效条件**。

- **C1**：一套在 4 GB 上运行的 MoE 执行栈，包含分级存储、流式专家供给和 fused packed-weight 内核，
  使 9.65 GiB 权重在 2.23 GiB 峰值下完成推理（§4.1，§5.2）。
  **同一设备上热态 llama.cpp 为 23.7–25.5 tok/s（ECS 的 8–9 倍），
  但常驻内存 3.00–3.34 GiB vs 5.37 GiB**（§5.6 G1）；
  C1 主张的是**可行性**与**资源占用**，不是吞吐竞争力。
- **C2**：在双卡上用 ECS 完成专家分片执行，远端 rank 不加载 transformer，token 与单卡完全一致（§4.3，§5.3）。**注意**：该实验验证的是跨设备分片的可行性与协议正确性，
  其性能提升主要来自显存总量的增加，不能归因于调度本身（§5.3.1）。
- **C3**：在 ECS 之上实现 paged KV slot allocator 与前缀块共享，并给出它们的**适用范围边界**（§4.4，§5.4，§6.1）。
- **C4**：三项否定性结果，包含可复现的失效条件与根因分析（§6）。

此外，§6 用一整节报告否定性结果，因为我们认为它们比正面结果更容易被误判。

---

2. 背景与相关工作

2.1 三条技术线

MoE 推理。 Switch Transformer 与 GShard 确立了稀疏激活的基本形式：router 为每个 token 选择 k 个专家，专家并行执行后加权合并。DeepSpeed-MoE 讨论了专家并行与通信开销。

Mixtral 与 DeepSeek-V2 把这条路推进到开源模型。DeepSeek-V2 的另一处不同是 MLA（Multi-head Latent Attention），它压缩 KV 而不牺牲太多质量，这对显存受限场景格外重要。

KV 缓存分页。 PagedAttention 把 KV 切成固定大小的 block，用 block table 做间接寻址，消除了连续分配带来的碎片与预分配浪费；vLLM 在此基础上实现了连续批处理；Orca 确立了迭代级调度（continuous batching）的语义。本文的 paged KV 沿用这套抽象，但需要适配 MLA 的 head 维度与 GGUF 权重格式。

显存受限的专家 offload。 近年来出现了一批直接针对"显存装不下专家"的工作：OD-MoE 在 <1 GB GPU 上做无缓存的按需专家加载；PreScope 用预测驱动的跨层调度生成预取计划；SPICE 用低秩专家代理（LoRE）配合置信度感知的 CPU-GPU 编排，在预测失误时避免同步重载停顿；Mira 用轻量逐层预测器提前两层预测专家并配合两级 GPU 缓存；D2MoE 在真实边缘设备上做调度；KTransformers 提供三级 KV 缓存与专家放置；MoE-Lightning 用 CGOPipe 把 offload 的权重读取与计算重叠起来。

2.2 与本文的关系

上述系统各有明确的单点贡献。ECS 的差异不在某个技术点——paged KV、专家 offload、连续批处理都有先例——而在**覆盖的层次与所报告的边界**：ECS 在同一个目标设备约束下同时处理权重供给、KV 管理、请求调度与跨设备协同，并且报告了每一项优化在该约束下的收益边界与失效条件。

表 1 按四个层面整理现有系统与 ECS 的覆盖情况。

表 1：现有系统与 ECS 的覆盖对比

系统	权重供给	KV 管理	请求调度	跨设备	目标硬件下限	否定性结果
OD-MoE	按需加载（无缓存）	—	—	—	<1 GB GPU	部分
PreScope	预测预取	—	跨层调度	—	数据中心 GPU	否
SPICE	LoRE 代理 + 编排	—	CPU-GPU 编排	—	边缘设备	部分
Mira	逐层预测 + 两级缓存	—	—	—	边缘设备	否
D2MoE	offload	—	设备级调度	—	边缘设备	否
KTransformers	专家放置	三级 KV	部分	CPU+GPU	单机大内存	否
MoE-Lightning	CGOPipe 流水	—	流水线	—	高带宽 CPU-GPU	—

系统	权重供给	KV 管理	请求调度	跨设备	目标硬件下限	否定性结果
ECS	**流式 + fused packed**	**paged + 前缀共享**	**连续批处理**	**双卡专家分片**	**4 GB GPU + 16 GB RAM**	**是（§6）**

需要说明的两点边界：

1. 该表对**其他系统**的归类依据其公开摘要与文档，未经我们复现验证；
   投稿前需逐条核对原始论文，避免误述他人工作。
2. ECS 覆盖四个层面不等于在每一层都优于专用系统。例如 od-MoE 在 <1 GB 场景下的
   显存效率、PreScope 的预测准确率，ECS 均未与之正面比较——这也是 §5.6 讨论的评测缺口。

3. 系统设计

3.1 分层存储

ECS 把专家权重放在三级存储里：

层级	位置	用途
L1	GPU 显存	当前 chunk 的 packed 专家权重、attention、共享专家
L2	CPU 内存（pinned）	预取槽位、专家缓存
L3	NVMe SSD	GGUF 全量权重，按需读取

对象粒度是 **expert chunk**（默认每 chunk 最多 8 个专家），而不是整层或整模型。
这个粒度的选择来自一个实测约束：整层 offload 会带来约 1.86 tok/s 的回退，
而专家粒度加冷热切分可以做到约 2.50 tok/s（§5.2）。

3.2 控制面与数据面分离

ECS 的控制器（`ECSController`，C++ 实现 heat/LRU/tier 策略）驻留在 CPU，负责：

- 路由热度的统计与持久化；
- 缓存的保留与淘汰决策；
- 跨设备的放置（`ECSRouter` 按队列深度与 KV 压力选择设备）。

GPU 侧只负责计算当前传入的专家。两者之间通过明确的回调接口通信，
使控制器可以独立测试，也可以在单测里用模拟的设备后端跑通。

3.3 流式专家供给

一次前向的执行序列：

1. router 产出每层的专家 ID 与 token 分配；
2. 按 chunk 切分，构造本次需要的 packed 权重读取请求；
3. 后台线程把下一 chunk 读入 pinned 槽位（双缓冲）；
4. 主线程做 H2D 拷贝，然后发 grouped kernel。

关键实现约束：**权重不做反量化展开。**fused kernel 直接消费 Q4_K/Q5_0/Q8_0 的 packed 行，
避免了"解量化到 FP16 → 读 HBM"的往返。这条改动是单卡路径从"能跑"到"能用"的分界点。

3.4 双卡专家分片

第二张卡（rank 1）是一个**不加载 transformer 的专家执行器**：它只打开 GGUF 中路由专家对应的字节区间，
接收激活值，执行 grouped gate/up/down，回传结果。协调卡（rank 0）保留 attention、共享专家和 LM head。

角色	持有	峰值显存
rank 0（协调）	attention + shared experts + LM head + 部分专家	2.21–2.28 GiB
rank 1（专家执行器）	仅专家权重与激活缓冲	**38–62 MiB**

3.5 请求调度

在 ECS 之上实现连续批处理：请求动态进入空闲 slot，完成后释放；KV 按 block 分配，
每个请求持有自己的 block 列表；解码时只重建 block table，不重排任何一行。

前缀共享通过 block 级累积 hash 实现：命中则 retain，之后只对后缀做前向，
写回时保证**已共享的 block 不可变**——block 对齐的 prompt 不会被交出最后一块，
因为那一块仍需写入本行尾部。

---

4. 实现

4.1 单卡执行栈（4 GB）

组件	实现
权重量化	W4A16 / W3A16（group 128；3-bit 把 8 个权重压进 3 字节）
GEMV	手写 CUDA，batch tile 8
prefill GEMM	32×64 CTA tile，CUDA core + FP32 累加（**未使用 Tensor Core**）
attention	原生 GQA decode kernel（static）+ paged MLA（双卡路径）
KV	静态 cache + CUDA Graph；paged block 池；FP16/INT8/FP8

组件	实现
调度	固定槽位（低延迟）与连续批处理（吞吐）两条路径

4.2 MoE 内核

三种 fused 路径：Q4_K、Q5_0、Q8_0。它们直接消费 packed expert 行，
每个 chunk 的 gate/up 与 down 各只发一次 kernel。作为回退保留 `--no-fused-moe` 的
"反量化 + grouped bmm"路径，用于数值对照。

4.3 传输层

协调卡与专家执行器之间使用 NCCL 点对点 request/reply：固定长度 JSON header + 激活张量。
一次请求对应一次 `isend`（header、tokens）与一次 `irecv`（输出）。

该协议的实现约束（重要）：协议是严格 FIFO、无请求 ID，worker 按到达顺序回包。
因此它只能由单一生产者驱动。我们后来把客户端改为「提交入队 + 单 I/O 线程排空」，
使调用方可以来自多线程而不破坏协议顺序（§6.2）。

4.4 Paged KV 与前缀共享

- `PagedKVCacheManager` 持有物理 block 池；每请求持有 `block_ids` 并按需逐块增长；
- decode 每步重建 `[batch, max_blocks]` block table；
- 两种 prefill：批量左填充 dense 前向后一次性写回（默认），或逐请求直接 paged 前向；
- 前缀共享仅在 paged prefill 路径可用（dense 批量路径从位置 0 写整行，无法从偏移开始）。

---

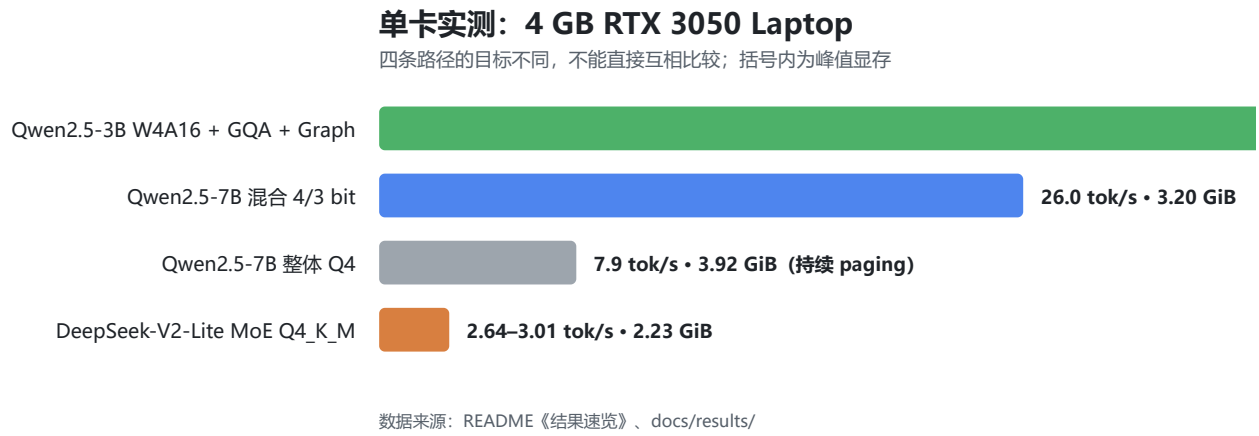
5. 评测

5.1 实验环境

配置	单卡	双卡
GPU	RTX 3050 Laptop 4 GB (SM86)	2× RTX 3080 Ti 12 GB（无 P2P）
CPU / RAM	Ryzen 5 5600H / 16 GB	Xeon Silver 4214R / 440 GiB
系统	Windows 11	Ubuntu 22.04
PyTorch / CUDA	2.5.1+cu121	2.5.1+cu121

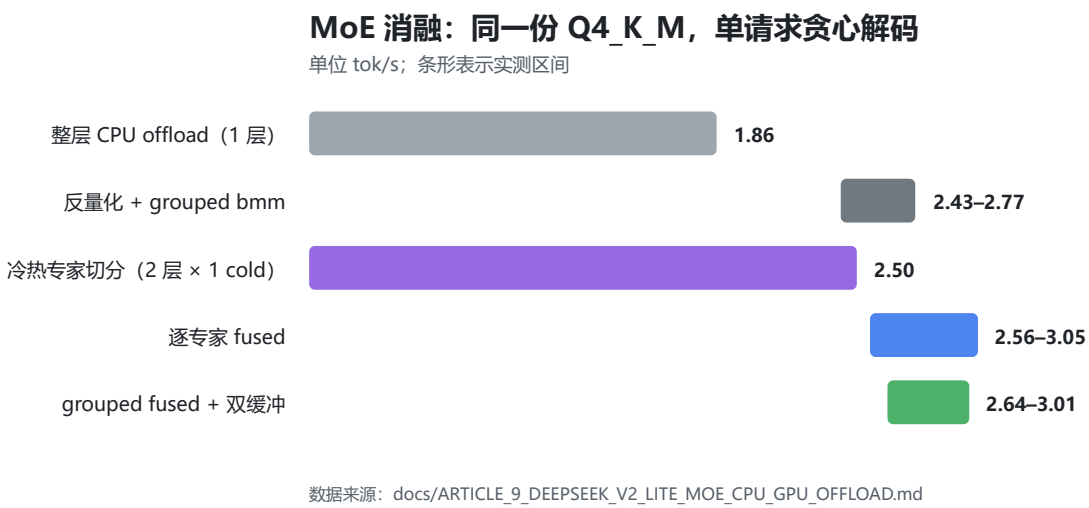
5.2 单卡：4 GB 上能跑到什么程度

模型	量化	峰值显存	解码
Qwen2.5-3B	W4A16 + GQA + Graph	2.49 GiB	**42.7 tok/s**（稳态）
Qwen2.5-3B（常驻网关）	W4A16 + 256 context	~2.5 GiB	44.8 tok/s / 0.895 s 每请求
Qwen2.5-7B	4-bit attention/embed + 3-bit MLP/head	3.20 GiB	**25.96 tok/s**（均值）
Qwen2.5-7B	整体 4-bit	3.92 GiB	7.94 tok/s（持续 paging）
DeepSeek-V2-Lite	Q4_K_M + grouped fused	2.23 GiB	2.64–3.01 tok/s



MoE 路径的消融（同一份 Q4_K_M、贪心解码、单请求）：

变体	解码
反量化 + grouped bmm	2.43–2.77 tok/s
逐专家 fused	2.56–3.05 tok/s
grouped fused + 双缓冲	**2.64–3.01 tok/s**
整层 CPU offload（1 层）	1.86 tok/s
冷热专家切分（2 层 × 1 cold）	2.50 tok/s

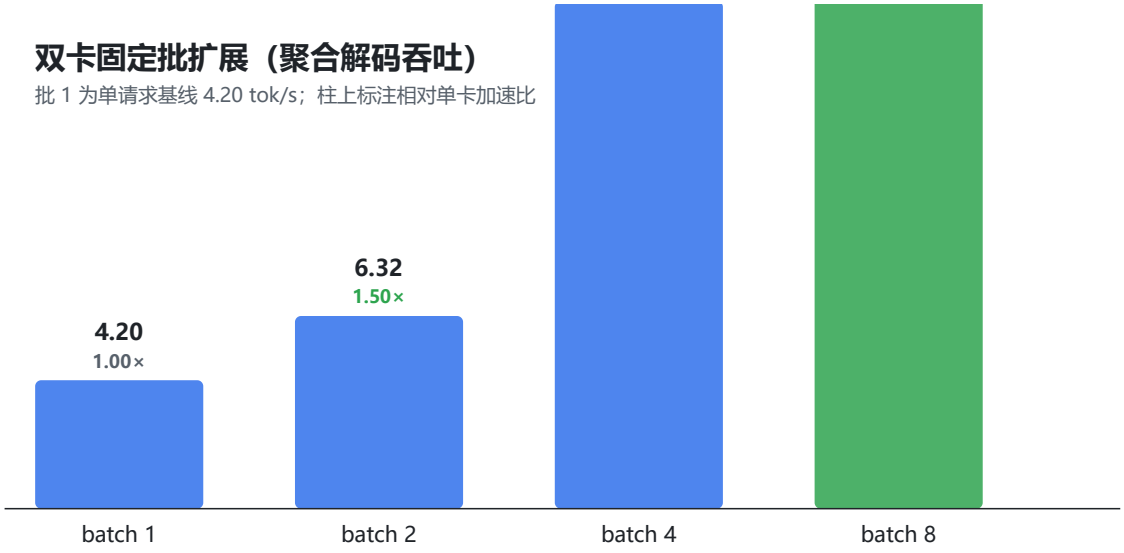


对照基线：同一份 Q4_K_M 在 llama.cpp CPU 路径为 9–13 tok/s。
因此单卡 MoE 路径的结论应表述为「在不 OOM 的前提下可运行」，而非性能优势。

5.3 双卡：专家分片与批处理

单请求与批量（贪心、短 prompt）：

路径	解码	相对单卡
单卡基线	4.20 tok/s	1.00×
双卡同步 NCCL 专家并行	3.44 tok/s	0.82×
双卡异步 NCCL	4.90 tok/s	1.17×
双卡混合（prefill 批量 + decode 异步）	4.71 tok/s	1.12×
固定 batch 2	6.32 tok/s（聚合）	1.50×
固定 batch 4	16.60 tok/s（聚合）	3.95×
固定 batch 8	**25.90 tok/s（聚合）**	**6.17×**



所有批大小的输出 token 与单卡基线完全一致

所有路径的输出 token 与单卡基线完全一致。

5.3.1 显存总量混淆（必须声明）

双卡配置的总显存是 24 GB，是单卡 4 GB 的 6 倍。因此 §5.3 中 6.17× 的聚合吞吐提升**主要来自显存总量的增加，而不是调度本身的贡献**：更大的显存使专家可以常驻，不再需要流式供给。

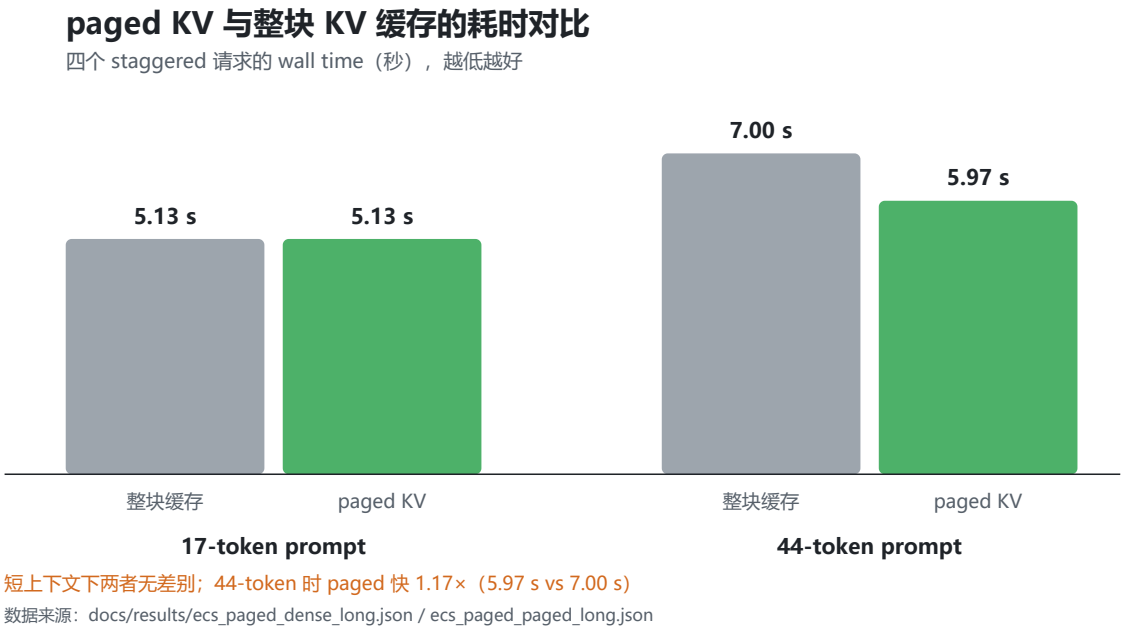
这一组实验验证的是**跨设备专家分片的可行性与协议正确性**（远端 rank 不加载 transformer、token 完全一致、心跳与监督重启可用），而不是"ECS 在显存受限下的调度收益"。

真正的调度收益应在单卡 4 GB 内部的消融中观察，即 §5.2 的 MoE 变体表（反量化 bmm 2.43–2.77 / 逐专家 fused 2.56–3.05 / grouped fused + 双缓冲 2.64–3.01 tok/s）与 §5.4 的 paged KV、前缀共享实验。

5.4 连续批处理、paged KV 与前缀共享

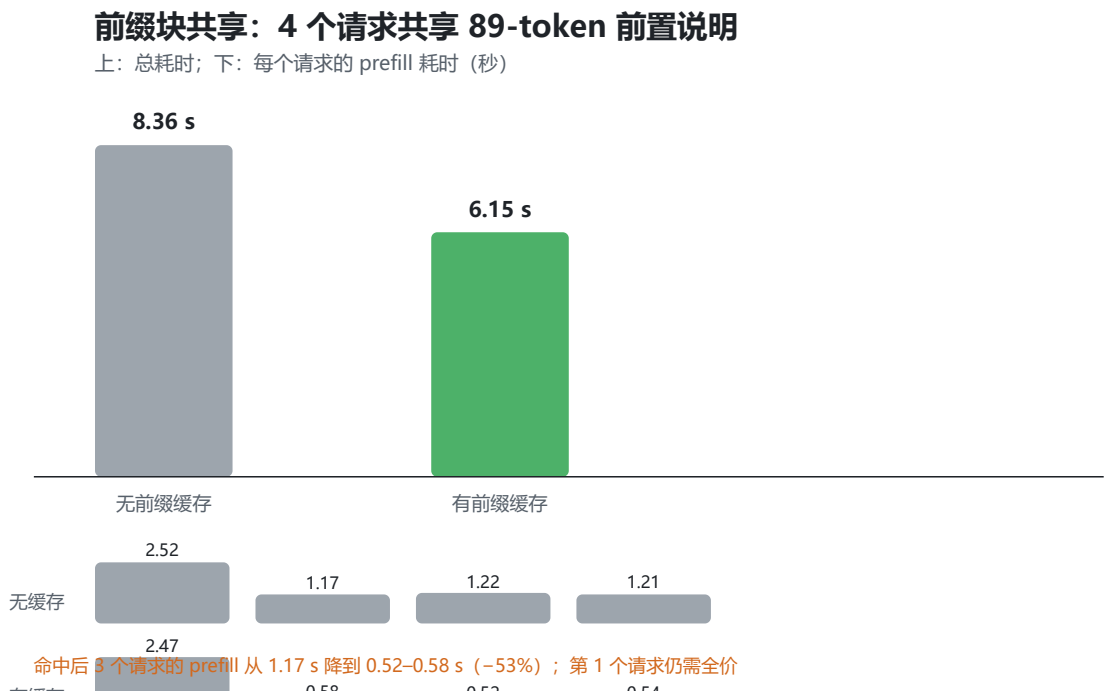
四个 staggered 请求（0.15 s 间隔），双卡：

配置	wall	聚合解码
dense slot cache，整批 admission	6.13 s	4.24 tok/s
paged KV，17-token prompt	5.13 s	5.07 tok/s
dense，同 workload	5.13 s	5.07 tok/s
paged KV，**44-token prompt**	**5.97 s**	**4.53 tok/s**
dense，44-token prompt	7.00 s	3.86 tok/s



前缀共享（4 个请求共享 89-token 前置说明）：

配置	wall	每请求 prefill
无前缀缓存	8.36 s	2.52 / 1.17 / 1.22 / 1.21 s
前缀缓存（复用 5 个 block）	**6.15 s**	2.47 / **0.58 / 0.52 / 0.54** s



5.5 正确性

- 单卡与双卡 token 完全一致（`[207, 20, 12, 17, 28, 18, 100001]`）；
- paged 引擎在容量为 1 的串行配置下复现单请求输出；

- 每次 paged 运行结束 `used_blocks = 0`，KV 池完整归还；
- 全量单元测试 127 项通过（含 allocator、前缀不可变约束、引擎端到端、传输分发）。

5.6 评测缺口（提交前必须补齐）

我们在投稿准备中确认了三处缺口，按优先级排列：

G1 • 横向基线：已测。两者各占一个区间，没有一方全面胜出。

在同一台 RTX 3050 Laptop 4 GB 上，用 Ollama 0.24.0 内置的 llama.cpp 后端
运行同一份 DeepSeek-V2-Lite Q4_K_M，贪心解码、8 个新 token：

指标	llama.cpp（mmap，专家在 CPU）	ECS（流式专家供给）
冷启动到首个 token	166.6 s（n=1）	49–81 s（n=3，不可比，见下）
峰值常驻内存（RSS）	5.37 GiB	**3.00 GiB**
峰值显存	**386 MiB**	2.23 GiB
稳态解码（页缓存热）	**23.7–25.5 tok/s**	2.64–3.01 tok/s
冷态解码	7.46 tok/s	—

由此得到三点结论：

1. **冷启动维度已撤回。**三次重复显示 ECS 的加载耗时稳定（18–20 s），
   但 warmup（4.4 / 4.5 / 13.1 s）随操作系统页缓存状态剧烈波动；
   Windows 上清理 standby 缓存需要管理员权限（`NtSetSystemInformation` 返回
   `STATUS_PRIVILEGE_NOT_HELD`），因此两侧的"冷启动"测量不在同一缓存状态下进行，
   不可比较。本节只保留不受页缓存影响的指标（吞吐、峰值 RSS、峰值显存）。
2. **热态下 llama.cpp 的吞吐是 ECS 的 8–9 倍**，这一项 ECS 无法竞争，
   原因是数据通路不同：llama.cpp 从 DDR4 读取（数十 GB/s），
   ECS 每 token 约 1.35 GB 的专家权重必须过 PCIe（实际约 6–8 GB/s）。
3. **但 8–9 倍只在热态成立。**冷启动时 llama.cpp 只有 7.46 tok/s，
   与 ECS 相差约 2.5 倍。把热态数字当作全场景结论是不准确的。
4. **ECS 在常驻内存上占优**：3.00–3.34 GiB vs 5.37 GiB（约 56–62%）。
   冷启动维度因上述缓存问题不作为结论。

因此本文的定位是**区间分工**而非性能竞争：一次性查询或内存紧张时 ECS 的启动代价与
内存占用更合适；持续会话且页缓存可热时 llama.cpp 全面胜出。
`num_gpu=8` 档在 4 GB 卡上挂死（3.48 GiB 驻留、三分钟无输出），说明该设备的
llama.cpp GPU offload 路径不可用，纯 CPU 是唯一稳妥选择。

该基线的局限：每项冷启动数据 n=1；ECS 该次运行的解码速率未成功记录（沿用了同配置此前的多次测量）；llama.cpp 在部分 prompt 上输出更长，tok/s 可比性有限。

G2 • 单次测量，无重复与置信区间。 双卡机的同配置重复运行波动约 8%（同一 dense 配置 6.33 s vs 5.83 s）。因此本文中 5% 以内的差异不应当作真实差异解读。定稿前需对每个数据点做 3–5 次重复并报告中位数与四分位距。

G3 • 双卡显存混淆。 见 §5.3.1，需要用同为 4 GB 的多卡环境或显存受限模式重新验证跨设备调度的独立贡献。

6. 否定性结果

6.1 paged KV 的收益边界

我们最初的假设是：整块 KV 缓存在 admission 时的重排与拷贝是吞吐瓶颈。
实现 paged 分配器后，实测推翻了这个假设：

prompt 长度	dense	paged
17 token	5.13 s	5.13 s
44 token	7.00 s	**5.97 s**

在短上下文下两者无法区分。拷贝 `[rows, heads, tokens, dim]` 只有在 `tokens` 变大时才是真实开销。真正的差距在 admission 同步和「每个调度迭代只推进一步 decode」。

6.2 prefill/decode 重叠在 NCCL 上不可行

我们尝试把 admission prefill 放到独立线程与独立 CUDA 流上。为此先把专家登台缓冲区（pinned 槽位、copy 事件、copy 流）改为按线程隔离——这是任何并发方案的前提。
随后在双卡上运行，进程以 `SIGABRT` 终止。

把提交路径改为「单 I/O 线程排空」以排除发送端交织后，问题依旧。结论是：

> 在 NCCL 2.x 的当前实现中，一个 `ProcessGroupNCCL` 的点对点通信运行在单条内部通信流上。
> 两个前向同时需要专家交换时，无论由哪个线程发起调用，都会竞争同一条流。

限定：这是 NCCL 实现层面的具体行为，可能随版本变化。本文观察到该现象的环境是 PyTorch 2.5.1+cu121 自带的 NCCL 2.x（精确版本号待补测）。若后续 NCCL 版本为每个 producer 提供独立通信流，本节结论需要重新验证。

这不是调度器层面的问题，客户端队列无法解决。

6.3 单 worker 是吞吐上限

把每个 chunk 整块发给某一个远端 worker，在只有一个 worker 时等价于一个串行队列。
扩展到 8 并发时，聚合吞吐受限于该 worker 的服务速率，而不是协调卡的调度能力。
复制 worker 是提高上限的必要条件；`PooledExpertClient` 已实现分发逻辑（按 in-flight 最少选择、回包归属、不平衡指标），但实验环境只有两张卡，**从未承载真实流量**，因此不构成本文的实证贡献。

---

7. 讨论

方法学教训。 本研究过程中出现三次判断错误，其中两次可归为「测量不足」，
一次可归为「叙事替代验证」：

1. 未经测量就假定拷贝是瓶颈（§6.1）；
2. 改动调度循环后未配对测量，一个 0.5 s 的阻塞把收益吃成 47% 的回退；
3. 把自设护栏抛出的 `ValueError` 误读为 NCCL 崩溃，并将其写入文档作为结论，
   事后才通过完整日志纠正。

第 3 条的特殊性在于：失败信号来自我们自己插入的断言，而不是被测系统。
我们因此建议：**跨会话恢复或长时间实验时，应交叉验证三类信息**——
会话日志（意图，可能错误）、版本控制历史（代码实际状态）、测试套件（当前是否可运行）。

适用边界的表述。 我们认为"在 X 显存上跑 Y 模型"这类陈述如果没有限定词是没有意义的。
本文所有数字都标注了路径（静态 Graph / paged 连续批处理 / 固定 batch），
因为它们的数量级差异恰恰来自路径选择而非实现质量。

---

8. 这项技术的价值，以及一个可以检验的预测

8.1 价值不在吞吐上

先说清楚：在本文的实验设备上，ECS 在吞吐上输给 llama.cpp 的 CPU 路径 8–9 倍（§5.6 G1）。
如果价值用 tok/s 来量，这个项目没有价值。它的价值在别处。

第一，它把"能不能跑"和"跑得多快"分开了。 主流框架默认显存不小于模型激活峰值，
于是小显存设备直接被排除在可行性讨论之外。ECS 证明 9.65 GiB 的 MoE 可以在 2.23 GiB
峰值显存下完成推理——速度慢，但**没有 OOM，且 token 与单卡完全一致。**
"不可行"和"不可用"是两回事，把前者变成后者是工程问题，把后者当前者判断则是认知错误。

第二，它给出了三项优化的失效边界（§6）。 paged KV 只在上下文变长时才回本；
prefill/decode 重叠被 NCCL 的单条内部通信流挡住，不是调度器能修的；
单 worker 是吞吐上限，复制 worker 是必要条件。这些结论对任何想在类似硬件上做同样事的人
都是可复用的负资产——他们不需要再花一轮去验证这三条。

第三，它记录了一次完整的失败诊断过程（§7）。 包括把自设护栏的异常误读为系统崩溃、
并将其写入文档的那一次。系统研究里失败实验的写作通常被省略，
我们认为这是该领域可复现性的一部分缺失。

8.2 为什么"装进手机"是一个有依据的方向，而不是口号

本文实测到的主要瓶颈是 **PCIe**：每 token 约 1.35 GB 的专家权重必须从主机内存
搬到独立显卡，PCIe 3.0 的实际带宽约 6–8 GB/s，于是吞吐被钉在 2.6–3.0 tok/s。
llama.cpp 之所以快 8 倍，是因为它在同一块 DDR4 内存里直接算，不付这笔过路费。

关键观察：这笔过路费是独立显卡架构的产物，不是 MoE 稀疏性的必然结果。

移动 SoC 与独立显卡的根本差别在于**统一内存**：

平台	计算单元与内存关系	有效带宽	是否需要"搬运"权重
笔记本 + 独立 GPU	分离，过 PCIe	6–8 GB/s	是
笔记本 + CPU 推理	统一	30–40 GB/s	否
手机 SoC（LPDDR5X）	**统一**	50–70 GB/s	否
手机 + UFS 4.0 闪存	—	2–4 GB/s	否（按页流式）

在统一内存架构上，ECS 的主要开销项直接消失：不需要 pinned staging、不需要 H2D 拷贝、
不需要双缓冲槽位。剩下的问题变成纯粹的数据供给：每 token 需要触达多少字节，
以及这些字节在 LPDDR5X 里还是在 UFS 闪存里。

按本文测得的口径（激活 2.4B 参数、Q4 约 4.5 bit/参数 ≈ 1.35 GB/token）：

- 若热门专家常驻 LPDDR5X（50 GB/s 量级），理论上限约 **30–35 tok/s**；
- 若全部走 UFS 4.0 流式读取（2–4 GB/s 实际带宽），约 **1.5–3 tok/s**——
  与本文在 PCIe 上实测到的水平相当，但**显存不再是约束**；
- 二者之间由专家热度分布决定，而这正是 ECS 的分级存储与 LRU 策略要解决的问题。

换句话说：**把本文的架构从"独立 GPU + PCIe"搬到"统一内存 SoC"，
限制它的那一项物理约束会消失。** 这不是承诺，是一个可以用现有部件拼出来的工程路径：
今天的旗舰手机已经有 12–16 GB LPDDR5X 与 256 GB–1 TB UFS 4.0，
而 DeepSeek-V2-Lite 这样的 15.7B MoE 在 Q4 下只有 9.65 GB。

8.3 一个可检验的预测

我们在此给出一个明确、可falsify的预测：

> **在五年内，一部 16 GB 统一内存的手机将能够在本地运行 15B 级别的 MoE 模型，
> 稳态解码不低于 5 tok/s，且不需要把模型完整常驻内存。**

它的依据不是乐观，而是四件事同时成立：

1. MoE 的稀疏性使每 token 只需要触达约 1.35 GB，而不是全部 9.65 GB；
2. 统一内存消除了本文实测到的主要瓶颈（PCIe 搬运）；
3. 专家路由存在明显的热度偏斜，因此分级缓存（本文的 L1/L2/L3）能命中大部分访问；
4. 手机的 LPDDR5X 带宽与 UFS 4.0 随机读性能已经进入上述计算所需的区间。

这个预测也给出了自己的证伪条件：如果**专家热度足够均匀**（缓存命中率长期低于 20%），
或者 **UFS 的随机读放大严重**（实际带宽远低于 2 GB/s），那么它不成立。
本文的 §6.3 已经显示，在单 worker 的情况下吞吐会被服务速率钉死——
移动端同样会遇到这个问题，只是串行化的对象从远端 worker 变成了闪存队列。

8.4 后续工作（按依赖排序）

1. **worker 复制与监督**：需要至少三个 rank；包含单 worker 心跳与死 worker 摘除。
2. **激活分片**：按专家归属切分 chunk，配每 worker 路由表，而非整块投递。
3. **collective all-to-all**：替换点对点 request/reply；这也是 prefill/decode 重叠的前置条件。
4. **在飞请求续跑**：当前 worker 崩溃导致整请求重试，KV 状态不恢复。
5. **统一内存后端**：把传输层抽象成后端接口，使 ECS 可以在 Apple Silicon、
   移动 SoC 等统一内存平台上跳过 staging 与 H2D 路径。这是 §8.2 论证的直接工程化。

8.5 写在最后

遗憾的是，受限于硬件水平，作者对于该框架的推进速度始终十分缓慢。

这不是客套。本报告中每一项结论背后都有一道具体的硬件闸门：

- 单卡实验全部在一张 4 GB 的 RTX 3050 Laptop 上完成，量化格式、KV 方案与调度策略
  的选择空间因此被反复压缩；
- 双卡的 RTX 3080 Ti 是临时借用的机器，**worker 复制始终无法验证**——
  §6.3 的池化分发器写完至今没有承载过一行真实流量；

这些不是辩解，而是这份报告的边界。我们用它们的代价换取了一件东西：
每一条结论都标注了它在什么条件下成立——包括那些不成立的。

附录 A：可复现命令

```
# 单卡 MoE 冒烟
PYTHONPATH=$PWD python scripts/smoke_deepseek_v2_lite_moe.py \
  --prompt "只回答数字：5-2=?" --max-new-tokens 8
```

```
# 双卡连续批处理（paged KV）
PYTHONPATH=$PWD torchrun --standalone --nproc_per_node=2 \
  scripts/bench_ecs_paged_continuous_batch.py \
  --max-batch-size 4 --max-new-tokens 8 --arrival-delay 0.15 --max-cache 256

# 双卡前缀共享
PYTHONPATH=$PWD torchrun --standalone --nproc_per_node=2 \
  scripts/bench_ecs_paged_continuous_batch.py \
  --max-batch-size 4 --max-new-tokens 8 --arrival-delay 0.15 --max-cache 256 \
  --prefill-mode paged --prefix-cache-blocks 64
```

附录 B：数据结构与结果文件

- 完整实验日志（含失败实验）：`docs/REMOTE_ECS_2X3080TI.md`
- 二卡定版范围：`docs/ECS_TWO_GPU_RELEASE.md`
- 结构化结果：`docs/results/ecs_*.json`
- 图（SVG，`scripts/make_figures.py` / `scripts/make_paper_figures.py` 生成）：`docs/figures/fig1..8_*.svg`
- 实现记录（十篇）：`docs/ARTICLE_1..10_*.md`

参考文献

1. Fedus W., Zoph B., Shazeer N. *Switch Transformers.* JMLR, 2022.
2. Lepikhin D., et al. *GShard.* ICLR, 2021.
3. Shazeer N., et al. *Outrageously Large Neural Networks.* ICLR, 2017.
4. Rajbhandari S., et al. *DeepSpeed-MoE.* ICML, 2022.
5. Kwon W., et al. *Efficient Memory Management for LLM Serving with PagedAttention.* SOSP, 2023.
6. Yu G.-I., et al. *Orca.* OSDI, 2022.
7. Dao T., et al. *FlashAttention.* NeurIPS, 2022.
8. DeepSeek-AI. *DeepSeek-V2.* arXiv:2405.04434, 2024.
9. Jiang A. Q., et al. *Mixtral of Experts.* arXiv:2401.04088, 2024.
10. Frantar E., et al. *GPTQ.* ICLR, 2023.
11. Lin J., et al. *AWQ.* MLSys, 2024.
12. OD-MoE: 面向极小显存的按需专家加载。（出处待核）
13. PreScope: 预测驱动的专家预取与跨层调度。（出处待核）
14. SPICE: 低秩专家代理与置信度感知的 CPU-GPU 编排。（出处待核）
15. Mira: 逐层专家预测与两级 GPU 缓存。（出处待核）

16. D2MoE: 边缘设备上的 MoE 调度。（出处待核）
17. KTransformers: 三级 KV 缓存与专家放置。（出处待核）
18. MoE-Lightning / CGOPipe: offload 专家权重读取与计算重叠。（出处待核）
19. *Not All Models Suit Expert Offloading*（路由一致性研究）。（出处待核）

> **参考文献核验状态**：第 1–11 条为通行文献，但年份、会议与卷期仍需逐条核对；
> 第 12–19 条来自同行评审意见中提到的相关工作，**我们尚未取得原文**，
> 仅用于说明相关工作覆盖面，投稿前必须逐条补全并确认其主张，
> 否则应删除对应表述（尤其表 1 中对其能力的归类）。